ClusterAI-HPC – Complete Beginner Guide and Project Roadmap

Kubernetes-Based Slurm Platform for AI Training, Inference, and High-Performance Computing — CDAC HPC-AI Project

1. The Project in Plain Words

You are building one website (a dashboard) that sits on top of three things:

Piece	What it does	Simple analogy
Slurm	Queues and runs heavy computing jobs on a group of machines	A ticket counter: you submit a job, it waits in line, then runs
Kubernetes	Runs your apps (Flask, database, Grafana) in containers and keeps them alive	A building manager for your apps
AI/ML	Trains a model on data, then predicts on new data	The "smart" part

The Flask backend is the middleman. The browser asks Flask, and Flask talks to Slurm, Kubernetes, the ML code and the database.

Do not try to build everything at once. Build in layers, where each layer works on its own, so you always have something to demo.

2. Problem Statement

HPC scheduling, cluster monitoring, AI training and data analysis normally need separate tools: a terminal for Slurm, kubectl for Kubernetes, notebooks for ML, and Grafana for graphs. A beginner or researcher cannot see the whole workflow in one place. ClusterAI-HPC provides one dashboard for submitting jobs, monitoring resources, training and using models, and comparing parallel performance.

3. Scope

In scope (what you will actually deliver)

- Dashboard with cluster and job status
- Slurm job submit, status and logs (from fixed job templates)
- One complete AI train-and-predict workflow
- Serial vs OpenMP vs MPI benchmark with speedup and efficiency
- Basic monitoring with Prometheus and Grafana
- Documentation with honest, measured results

Out of scope or optional

- CUDA (only if you get a compatible NVIDIA GPU)
- All four datasets (pick one or two first)
- Production-grade security and real multi-user login
- Multi-cloud deployment

4. Applications

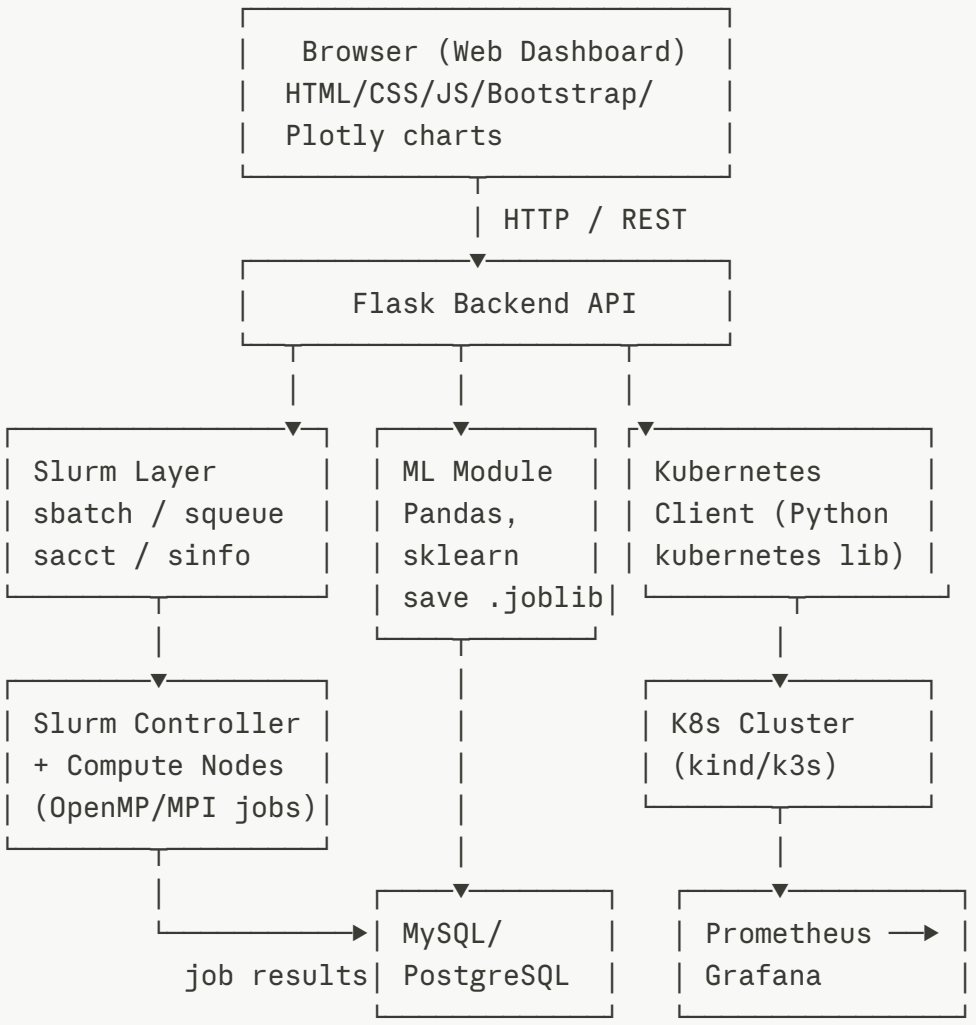
- University or lab HPC portals
- Teaching HPC to students
- Small-company ML job management
- Performance-comparison tools for parallel programs
- A base for MLOps and AI platform engineering work

5. Future Scope

- User login and roles
- GPU scheduling
- Auto-scaling of compute nodes
- Job usage and cost accounting
- Model registry (for example MLflow)
- PyTorch distributed training
- Email or Slack alerts on job failure

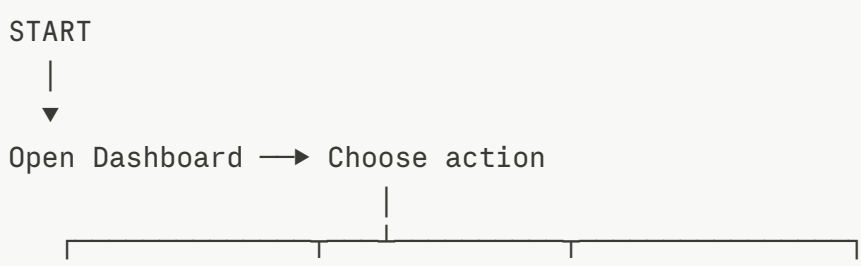
- Cloud deployment

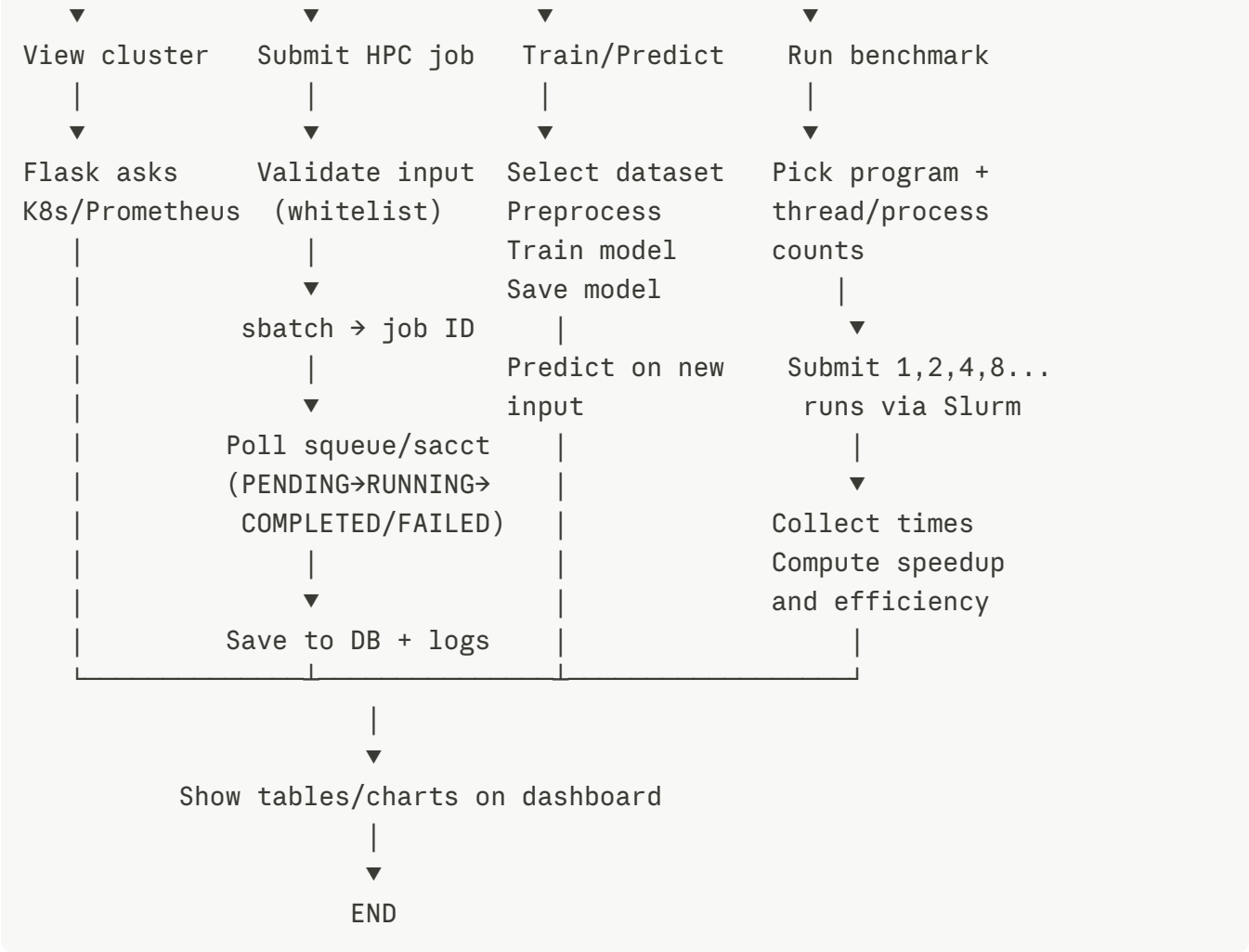
6. Block Diagram



7. Project Flow and Flowchart

The user opens the dashboard and chooses an action: view cluster, submit job, train model, run benchmark, or view reports.





8. Algorithms

8.1 Job submission and tracking

1. Receive the job type and parameters from the user.
2. Check the job type against a fixed list. Never run user text as a shell command.
3. Generate the .sbatch file from a template.
4. Run sbatch, capture the job ID, and save it in the database with status PENDING.
5. Every few seconds run squeue -j <id> (or sacct once it finishes) and update the database.
6. When the job is COMPLETED or FAILED, read the output file, store the runtime and show it on the dashboard.

8.2 Benchmark

1. For p in {1, 2, 4, 8}: run the same workload and record the time T(p).
2. Speedup: S(p) = T(1) / T(p)
3. Parallel efficiency: E(p) = S(p) / p
4. Plot time, speedup and efficiency curves.

Speedup flattens as p grows. Explain it with Amdahl's Law: $S = 1 / ((1 - f) + f/p)$, where f is the parallel fraction of your program.

8.3 ML workflow

1. Load the CSV and clean it (missing values, encode categories, scale).
2. Split into train and test sets (80/20).
3. Train a model (start with RandomForest or Linear Regression).
4. Evaluate: accuracy or F1 for classification; MAE, RMSE or R^2 for regression.
5. Save the model with joblib.dump.
6. For inference: load the model, preprocess the new input the same way, predict, and return the result.

9. Roadmap (about 12 weeks, adjust to your deadline)

Weeks	Phase	Goal	Done when
1–2	Foundations	Linux commands, Git, Python venv, Docker basics	You can build and run a container
3	Flask + DB	Flask app, MySQL/PostgreSQL tables, simple dashboard page	Page shows data from the DB
4–5	ML workflow	One dataset: train, save, predict, show on page	Predict button works
6–7	Slurm	Working Slurm (single node first), submit a job from Flask	Job ID and status show on dashboard
8	Benchmark	OpenMP and MPI programs run through Slurm, charts	Speedup graph from real runs
9–10	Kubernetes	Flask and DB in k8s, cluster page, then try Slinky	Pods visible in dashboard

Weeks	Phase	Goal	Done when
11	Monitoring	Prometheus + Grafana	Live CPU/memory graphs
12	Polish	Testing, documentation, demo, PPT	Rehearsed demo

Recommended MVP order: dashboard, then Slurm job management, then one ML workflow, then real benchmarks. Add the remaining dataset modules and advanced Kubernetes–Slurm integration only after those four work.

Suggested team split (five members)

1. Frontend and charts
2. Flask backend and database
3. Slurm and benchmark programs
4. Kubernetes and monitoring
5. ML module and documentation

Everyone should understand the full flow, because the guide may ask any member about any part.

10. Step-by-Step Build

Step 0 – Environment

Start small. Use a Linux laptop or an Ubuntu VM with 8 GB RAM or more and 4 or more CPU cores. Install Docker, Python 3.10+, Git, gcc, and mpich or openmpi. Ask your guide what lab nodes or GPUs are available, because that decides what is realistic.

Step 1 – Learn just enough

- Linux: ls, cd, cp, grep, chmod, ps, top, ssh, systemctl
- Docker: Dockerfile, docker build/run, docker compose
- Slurm: sinfo, sbatch, squeue, scancel, sacct
- Kubernetes: pod, deployment, service, kubectl get/describe/logs

Step 2 – Flask skeleton

```
from flask import Flask, jsonify
app = Flask(__name__)

@app.route("/api/health")
def health():
    return jsonify(status="ok")
```

Add routes as you go: /api/jobs, /api/cluster, /api/train, /api/predict, /api/benchmark.

Step 3 – Database tables (starter)

- jobs(id, slurm_job_id, type, params, status, submit_time, end_time, runtime_sec, log_path)
- models(id, name, dataset, algo, metrics_json, file_path, created_at)
- predictions(id, model_id, input_json, output_json, created_at)
- benchmarks(id, program, nproc, run_time, speedup, efficiency, created_at)

Step 4 – ML workflow

Pick one dataset first (CPU workload is the easiest to get or generate). Do it in a notebook first, then move the code into ml/train.py and ml/predict.py and call them from Flask.

Step 5 – Slurm

Start with single-node Slurm on your VM, or a Slurm-in-Docker setup. A minimal job script:

```
#!/bin/bash
#SBATCH --job-name=bench_omp
#SBATCH --cpus-per-task=4
#SBATCH --output=logs/%j.out
export OMP_NUM_THREADS=4
./bench_omp
```

Flask calls it with subprocess.run(["sbatch", path], capture_output=True). Always pass a list (not a shell string) and only allow templates you wrote.

Step 6 – Benchmark programs

Write one compute-heavy task (matrix multiplication, or Monte Carlo pi) in three versions:

- Serial C
- OpenMP: add #pragma omp parallel for reduction(+:sum) and compile with gcc -fopenmp
- MPI: use MPI_Init, split the work by rank, combine with MPI_Reduce, compile with mpicc and run with mpirun -np 4

Run each version several times and average. Keep the problem size the same across versions. Then compute speedup and efficiency.

Step 7 – Kubernetes

Install kind or k3s. Containerize Flask and the database, then write Deployment and Service YAML. Use the Python kubernetes client in Flask to list nodes and pods for the cluster page.

Step 8 – Kubernetes + Slurm integration

This is the hardest part, so do it last. Try the Slinky slurm-operator (installed with Helm) on your Kubernetes cluster, following its current official documentation, since versions and requirements change. Have a fallback ready: run Slurm in a separate VM or container set, and let Flask in Kubernetes reach it over SSH or slurmrestd. You can then truthfully say "integration investigated; working approach is X", which your synopsis already allows.

Step 9 – Monitoring

Install Prometheus and Grafana (the kube-prometheus-stack Helm chart is the common route). Embed Grafana panels in your dashboard, or query Prometheus from Flask and draw Plotly charts.

Step 10 – Test, document, demo

Write test cases: job success, job failure, wrong input, model file missing. Capture screenshots and prepare a 5-minute demo script.

11. Pros and Cons

Pros	Cons / risks
Covers HPC, parallel programming, Kubernetes, ML and web in one project	Large scope for a beginner, so scope control is critical
Very relevant to industry (HPC, MLOps, platform engineering)	Slinky / Slurm-on-Kubernetes setup can be difficult and time-consuming
Works with partial features, which makes it good for demos	CUDA needs a compatible NVIDIA GPU
Real measured benchmarks are convincing	Small VMs show poor scaling, so results may look weak (explain why)
Strong portfolio and interview material	Subprocess/SSH job submission is risky if not whitelisted

12. Things You May Have Missed

- Hardware reality: confirm whether you get GPUs or multiple nodes. If not, state that CUDA is planned or future work.
- Honest results: your synopsis promises that all numbers come from real tests and unfinished features are labelled. Follow that strictly and never paste invented numbers.
- Security basics: input validation, no shell injection, secrets in environment variables and not in code.
- Version control: one GitHub repository with a good README and folders or branches per module.
- Logging and error handling: job failures must show clearly on the dashboard.
- Documentation list: architecture diagram, setup guide, user manual, test report, limitations, results and screenshots.

Likely viva questions

- What is the difference between Slurm and Kubernetes, and why use both?
- What are speedup and parallel efficiency?
- Why is the measured speedup less than ideal?

- OpenMP vs MPI: when do you use each?
- How does your Flask backend talk to Slurm?
- What did not work, and why?

13. Your Next Three Actions

1. Ask your guide what hardware and cluster access you will have.
2. Set up Ubuntu, Docker and Python, and get the Flask /api/health route running.
3. Run a single sbatch hello-world job and see it in squeue.